

Game of The Amazons

Amazons - Project Description

Course: Artificial Intelligence

Professor: Prof. Dr. Izudin Džafić

Students: Emin Hadžiabdić, Armin Memišević, Muhammed Pašić

Academic year: 2025/26

Student IDs: 19960, 20016, 20109

Sarajevo, February 15th 2026

Table of Contents

Project Overview.....	2
Proposed vs. Delivered.....	2
Logic Implementation.....	3
Algorithm + Heuristics Implementation.....	4
Board Size and Style Implementation.....	6
Game Modes and Difficulties Implementation.....	7
Team Responsibilities and Task Distribution.....	8

Project Overview

The **Game of the Amazons** is a strategy board game for two players, traditionally played on a 10x10 board (though our version supports 6x6 and 8x8). It is a "territory-capture" game with no pieces removed from the board.

Proposed vs. Delivered

Feature	Initial Proposal	Final (+additionally implemented)
Logic/Intercation	Interactive Queens, Highlights for valid moves, Selection of moves.	Implemented + <u>Undo Move</u> + <u>Move History</u> + <u>Rules Page</u>
AI Algorithm	Minimax + Alpha-Beta.	Implemented + <u>Stop AI feature</u>
Heuristics	Mobility, Territory, Spatial	Implemented
Difficulty Levels	Easy, Medium, Hard	Implemented
Board Size	10x10, 8x8, 6x6	Implemented
Game Modes	Player vs AI	Implemented + <u>PvP</u> , <u>AIvsAI</u> , <u>Animation Speed</u>
Themes	2 Styles (Wooden/B&W).	Expanded: 8 Styles + <u>Theme Creator</u> with color pickers.
Accessibility	2 Languages (EN/BS).	Expanded: 6 Languages (EN, BS, DE, ES, FR, JP).
Animations	Shooting Arrow, Field explosion, Moving Queens.	Implemented
Sound Effects	Animations + Win/Loss	Implemented

Logic Implementation

In the initial proposal, the scope of the logic was limited to interactive Queen movement, move validation, and basic selection. The final implementation significantly expanded this foundation to include a robust state-management system that supports **Undo functionality**, a detailed **Move History Log**, and an interactive **Rules Page**.

Core Logic & Validation: The game rules are encapsulated in the `Rules.h` file. We implemented a "Ray-Casting" approach for move generation. Since Queens and Arrows move identically to Chess Queens, the system scans all 8 directions (orthogonal and diagonal) to collect reachable tiles. A move is only valid if both the Queen's path to the destination and the Arrow's path from that destination are unobstructed.

```

1  // Full legality check for a move
2  inline bool isMoveLegal(const GameState& state, const Move& move) {
3      // Cannot move if game ended
4      if (state.isFinished())
5          return false;
6
7      // Must be correct player's turn
8      if (move.player != state.currentPlayer())
9          return false;
10
11     const auto& board = state.board();
12
13     // All positions must be inside the board
14     if (!board.isInsideBoard(move.queenFrom.row, move.queenFrom.col) ||
15         !board.isInsideBoard(move.queenTo.row, move.queenTo.col) ||
16         !board.isInsideBoard(move.arrow.row, move.arrow.col)) {
17         return false;
18     }
19
20     // Starting tile must contain player's queen
21     auto expectedTile = tileForPlayer(move.player);
22     if (board.getTile(move.queenFrom.row, move.queenFrom.col) != expectedTile)
23         return false;
24
25     // Queen movement must be valid
26     auto queenTargets = gatherReachableTiles(board, move.queenFrom);
27     if (!containsPosition(queenTargets, move.queenTo))
28         return false;
29
30     // Simulate queen move to validate arrow move
31     Board simulatedBoard = board;
32     simulatedBoard.setTile(move.queenFrom.row, move.queenFrom.col, TileContent::Empty);
33     simulatedBoard.setTile(move.queenTo.row, move.queenTo.col, expectedTile);
34
35     auto arrowTargets = gatherReachableTiles(simulatedBoard, move.queenTo);
36     if (!containsPosition(arrowTargets, move.arrow))
37         return false;
38
39     return true;
40 }

```

State Management (Undo & History): To support the "Undo" feature (which was not in the original proposal), we implemented a state-stack architecture in `GameState.h`. Every time a move is committed, the `Move` object is pushed into a `_moveHistory` vector. This allows the game to revert to any previous state by popping the last move and refreshing the board. This history is also visualized in the **Logs View**, providing players with a text-based record of the match (e.g., "Queen (4,1) -> (4,4), Arrow -> (5,5)").



```
1 void recordMove(const Move& move) { _moveHistory.push_back(move); }
2 [[nodiscard]] const std::vector<Move>& moveHistory() const { return _moveHistory; }
```

Algorithm + Heuristics Implementation

The core requirement was a Minimax algorithm with Alpha-Beta pruning. We delivered this but added critical enhancements for performance and user experience, specifically the **Stop AI** feature and advanced heuristic tiers.

Search Engine & Safety: The AI runs on a separate thread to keep the UI responsive. We implemented a safety mechanism using `std::atomic_bool _cancelAi` in `MainView.h` and `Algorithms.h`. This allows the user to interrupt the AI immediately (e.g., by clicking "Stop AI" or "New Game") without crashing the application, a feature essential for handling the high branching factor of 10x10 games.



```
1 // UPDATED: Propagates 'difficulty' to evaluate
2 inline int minimax(GameState& state, int depth, int alpha, int beta, Player maximizingPlayer,
3   Player perspective, std::size_t moveCap, Difficulty difficulty, const std::atomic_bool* cancel = nullptr) {
4
5   if (cancel && cancel->load()) throw SearchCanceled();
6
7   bool terminal = depth == 0 || detail::isTerminal(state);
8   if (terminal) {
9     GameState evalState = state;
10    evaluateWinState(evalState);
11    return detail::evaluateTerminal(evalState, perspective);
12  }
13
14  Player current = state.currentPlayer();
15  bool isMaximizing = current == maximizingPlayer;
16  auto moves = generateMovesForPlayer(state, current, moveCap);
17
18  if (moves.empty()) {
19    GameState evalState = state;
20    evaluateWinState(evalState);
21    return detail::evaluateTerminal(evalState, perspective);
22  }
23  ....
```

Heuristics (Intelligence Layers): We moved beyond simple mobility counting by implementing three distinct evaluation strategies based on difficulty :

1. **Mobility (Easy):** Simply counts available moves.
2. **Spatial Influence (Medium):** Calculates a score based on how close pieces are to the center and how much board area they "influence" or shadow.
3. **Territory Control (Hard):** Implements a "flood-fill" algorithm to determine which parts of the board are reachable only by the AI. This allows the Hard bot to trap opponents by sealing off entire sections of the board.

```

1  // UPDATED: Accepts 'difficulty' to toggle heuristics
2  inline int evaluate(const GameState& state, Player perspective, Difficulty difficulty) {
3      if (state.isFinished()) {
4          return detail::evaluateTerminal(state, perspective);
5      }
6
7      Player opponent = detail::getOpponent(perspective);
8
9      // HEURISTIC WEIGHTS
10     const int mobilityWeight = 3;
11     const int spatialWeight = 1;
12     const int territoryWeight = 5;
13
14     // 1. MOBILITY (All Difficulties)
15     int mobility = detail::mobilityCount(state, perspective) - detail::mobilityCount(state, opponent);
16     int score = mobility * mobilityWeight;
17
18     // Easy: Mobility Only (Fastest)
19     if (difficulty == Difficulty::Easy) {
20         return score;
21     }
22
23     // 2. SPATIAL INFLUENCE (Medium & Hard)
24     int spatial = detail::spatialInfluenceScore(state, perspective);
25     score += spatial * spatialWeight;
26
27     // 3. TERRITORY CONTROL (Hard Only)
28     // Flood fill is expensive, so we strictly limit it to Hard
29     if (difficulty == Difficulty::Hard) {
30         int territory = detail::territoryScore(state, perspective);
31         score += territory * territoryWeight;
32     }
33
34     return score;
35 }

```

Board Size and Style Implementation

The proposal listed three board sizes and two basic themes. The final product expanded this significantly to include **8 distinct themes** and a **Custom Theme Creator**, providing a highly polished visual experience.

Dynamic Board Geometry: The board rendering in [AmazonsBoardCanvas.h](#) is mathematically dynamic. Regardless of whether the user selects 6x6, 8x8, or 10x10, the [computeBoardGeometry](#) function calculates the optimal cell size to fit the window while maintaining a perfect square aspect ratio. This ensures the game looks correct on any screen size.

```

1  inline void AmazonsBoardCanvas::computeBoardGeometry() {
2      if (!_state) {
3          return;
4      }
5      int dim = _state->board().dimension();
6      if (dim <= 0) {
7          return;
8      }
9      double available = std::min(_viewSize.width, _viewSize.height) - 2.0 * _padding;
10     _cellSize = available / dim;
11     double width = _cellSize * dim;
12     double left = (_viewSize.width - width) / 2.0;
13     double top = (_viewSize.height - width) / 2.0;
14     _boardRect = gui::Rect(left, top, left + width, top + width);
15 }

```

Theme Engine: We implemented a flexible resource loading system. Users can switch between image-based themes (Wooden, Ice, Stone, etc.) and color-based themes (Bubblegum, Black & White). The standout addition is the **Custom Theme**, which uses the [SettingsPopup](#) to pass user-selected [ColorIDs](#) to the canvas. This allows users to mix and match any color combination for light and dark tiles using the integrated color pickers.

```

1  inline void AmazonsBoardCanvas::drawBoardGrid() const {
2      const auto& board = _state->board();
3      int dim = board.dimension();
4      for (int row = 0; row < dim; ++row) {
5          for (int col = 0; col < dim; ++col) {
6              auto rect = cellRect({row, col});
7              bool light = ((row + col) % 2) == 0;
8
9              // Image-based themes
10             if (_boardStyle == Style::Wooden || _boardStyle == Style::IceTheme ||
11                 _boardStyle == Style::StoneTheme || _boardStyle == Style::DiamondTheme ||
12                 _boardStyle == Style::TournamentTheme) {
13                 const gui::Image& tileImage = light ? _lightTileImage : _darkTileImage;
14                 if (tileImage.isOK()) {
15                     // Use simpler draw for better performance
16                     tileImage.draw(rect);
17                 }
18             }
19             ...color based themes implementation

```

Game Modes and Difficulties Implementation

We exceeded the standard Player vs. AI requirement by implementing a fully flexible "Player Type" system. This allows for **PvP** (local multiplayer), **PvA** (standard), and **AvA** (Spectator Mode).

Player Type Architecture: In `MainView.h`, we track `_whitePlayerType` and `_blackPlayerType` (Human or AI). The game loop checks these flags after every move. If the current player is AI, it triggers the background thread; otherwise, it enables mouse interaction for the human. This architecture also supports the added **Animation Speed** feature, allowing users to slow down AI matches (up to 3000ms delay) to observe the strategy in real-time.

```

1  inline void MainView::handleHumanMove(const Move& move) {
2  ... rest of implementation
3
4      if (isCurrentPlayerAI()) {
5          // If board is animating (arrow in flight / impact pending), defer AI until animations complete
6          if (_boardCanvas.isAnimating()) {
7              _boardCanvas.setAnimationFinishedHandler([this]() {
8                  // ensure this runs on the main thread and that AI still should move
9                  gui::thread::asyncExecInMainThread([this]() {
10                     if (isCurrentPlayerAI() && !_aiThinking && !_state.isFinished()) {
11                         requestAiMove();
12                     }
13                 });
14             });
15         }
16         else {
17             requestAiMove();
18         }
19     }
20     else {
21         _boardCanvas.setInteractionEnabled(true);
22         updateStatusForPhase(_boardCanvas.currentPhase());
23     }
24 }
```

Difficulty Scaling: The difficulty levels (Easy, Medium, Hard) are not just arbitrary labels; they fundamentally change how the engine behaves. We implemented **Variable Depth Logic** in `Algorithms.h`.

- **Easy:** Shallow search (Depth 1-2).
- **Medium:** Standard depth (Depth 3-4).
- **Hard:** Deep search (Depth 5+) with a "Variable Depth" optimization that searches only the top 6 most promising moves deeply, while shallow-searching the rest to save time.


```
1 inline Move getBestMove(const GameState& state, Difficulty difficulty, const std::atomic_bool* cancel = nullptr) {
2 ... rest of implementation
3
4     for (std::size_t idx = 0; idx < scored.size(); ++idx) {
5         const auto& entry = scored[idx];
6         GameState next = rootState.clone();
7         applyMove(next, entry.move);
8
9         int depthForMove = primaryDepth;
10        if (difficulty == Difficulty::Hard && idx >= deepSlots) {
11            depthForMove = shallowDepth;
12        }
13
14        int score = minimax(next, depthForMove, std::numeric_limits<int>::min(),
15                            std::numeric_limits<int>::max(), maximizingPlayer, perspective, moveCap, difficulty, cancel);
16
17        if (score > bestScore) {
18            bestScore = score;
19            bestMove = entry.move;
20        }
21    }
22
23    return bestMove;
24 }
```

Team Responsibilities and Task Distribution

- **Emin Hadžiabdić**
 - Development of the graphical user interface (GUI) using **natID**
 - Integration of GUI elements with the game engine
 - Ensuring smooth user interaction and visual consistency
- **Muhammed Pašić**
 - Implementation of core game rules and logic
 - Validation of legal moves, board updates, and turn management
 - Collaboration with engine developer to ensure correct rule execution
- **Armin Memišević**
 - Development of the main game engine
 - Implementation of the **Minimax algorithm** and **alpha-beta pruning**.
 - Coordination with rule and GUI components to ensure full game functionality